HPC第三关实验报告

1. 代码理解与接口

- 原始程序的输入、计算、校验、计时、输出分别在哪里?
  - 输入： 在 `main` 函数中通过 `generate` 或 `load` 生成并加载数据到内存。
  - 计算： 核心筛选循环（`for` 循环 + 短路求值 `if` 判断）被严格限制在 `core_ms` 计时区域内。
  - 校验： 通过 `reference_filter` 生成标准答案， 对比 `matched` 数量。
  - 计时： 使用 `std::chrono::steady_clock` 严格测量筛选循环耗时， 数据加载和输出不计入核心时间。
  - 输出： 筛选完成后， 将结果写入 `output` 数组并保存。
- 抽取了什么接口? 保持了哪些语义?
  - 接口抽取： 根据 `TASK.md` 要求， 理想接口为 `std::size_t filter_records(const Record* input, std::size_t count, Record* output, const Query& query);`
  - 保持语义： 严格保持输入顺序（稳定筛选），所有字段完整保留（`Record` 结构体完整拷贝），重复项不去重。

2. 环境与可复现基线

- 机器： Ubuntu 26.04.1 LTS (WSL 2)
- 开发环境： g++ 15.2.0,Python3.14.4,gcc15.2.0,cmake4.2.3
- 处理器： AMD Ryzen 9 8940HX with Radeon Graphics，x86-64架构，16核32线程
- GPU：NVIDIA GeForce RTX 5060 Laptop，CUDA Version:13.1
- 编译命令：
  - Baseline: `g++ -std=c++17 -O2 -o baseline baseline.cpp`
  - Optimized: `g++ -std=c++17 -O3 -march=native -o optimized optimized.cpp`
- 运行公开正确性测试
  - python3 test_public.py --binary ./baseline
  - 跑两组，第一组命中率为50%（好像是默认的）
  - ./baseline
  - ./baseline --count 100000 --hit-percent 1 --seed 2026
- 优化
  - 修改baseline。cpp文件，并改名成optimized.cpp

for (std::size_t i = 0; i < input.size(); ++i) { if ((input[i].flags & query.required_flags) == query.required_flags && input[i].latency_us >= query.min_latency_us && input[i].timestamp >= query.begin && input[i].timestamp < query.end) { output[matched++] = input[i]; } }

// 尝试把开销低、过滤效果最明显的条件放在最前面

// 位运算非常快，且 required_flags 一般能过滤掉一部分数据

  - 编译 g++ -std=c++17 -O3 -o optimized optimized.cpp
  - 跑一下正确性测试 python3 test_public.py ./optimized
  - 再跑 ./optimized
  - ./optimized --count 100000 --hit-percent 1 --seed 2026

大概就这样，因为不是边打边记录，而ai已经乱完了，力竭了

- 线程数： 1

- 计时边界： 核心筛选 `core_ms`，不包括数据生成/加载和最终输出。

实现	N	命中率/分布	线程数	5次以上原始耗时 (ms)	中位数 (ms)	输出数量
原始（Baseline）	100000	50% / random	1	0.3359, 0.3079, 0.2993, 0.3006, 0.3277	0.3079	50156
优化 (Optimized)	100000	50% / random	1	0.3016, 0.2976, 0.2898, 0.3019, 0.2970	0.2976	50156
原始（Baseline）	100000	1% / random	1	0.3468, 0.3278, 0.3224, 0.3189, 0.3184	0.3224	995
优化 (Optimized)	100000	1% / random	1	0.3471, 0.3455, 0.3259, 0.3227, 0.3260	0.3260	995



3. 热点、假设与选择

- 热点证据：在中等命中率（50%） 下， 主循环中的 `if` 条件判断成为主要瓶颈， 由于数据分布随机，CPU的分支预测器频繁猜错， 导致流水线清空。
- 比较至少两个优化候选：
  1. 调整 `if` 短路求值的条件顺序， 将开销低、过滤效果明显的条件前置， 减少无效比较。
  2. 使用 `-O3 -march=native` 让编译器生成针对本地 CPU 的 SIMD 优

化指令。

- 选择依据： 上述两种方案均不改变基础语义 ， 且预期能减少分支预测失败带来的惩罚。

4. 实现与正确性

- 实现关键点： 修改了 `optimized.cpp` 中核心循环的 `if` 条件顺序（例如将`flags` 和 `latency_us` 提前），并调整编译参数为 `-O3 -march=native`。
- 稳定顺序保证：筛选结果依然严格按照输入顺序写入 `output[matched++]`。
- 临时内存与同步： 单线程执行 ， 无同步开销。
- 公开测试命令及结果： `python3 test_public.py ./optimized`，结果 `Ran 7 tests in 0.211s OK`。
- 边界用例： 保留了与 Baseline 一致的 `timestamp` 区间判断（如 `<` 和 `<=` 的区别），输出 `matched` 数量与基线完全一致。

5. 性能与适用条件

- 按相同口径比较：
  - 在 50% 中等命中率下 ， 优化版中位数从 0.3079ms 降至 0.2976ms ， 加速比约为 **1.035** 倍。
  - 在 1% 极低命中率下 ， 优化版中位数从 0.3224ms 变为 0.3260ms ， 出现了退化。
- 退化原因分析： 极低命中率下 CPU 分支预测近乎完美 ， 流水线几乎无停顿 ， 额外的比较运算成为了新的负担。
- 适用条件：适用于中等命中率（约 20% - 80%） 的筛选场景。
- 退化场景：不适用于极低命中率（ <5%）或数据量极小（无法分摊编译优化带来的指令膨胀） 的场景。

6. 复盘

- 记录错误与验证修复： 在优化初期出现过 `error: validation : wrong count`，原因是不小心改变了边界符号（如把 `<` 写成 `<=`）。对比 `support.hpp` 中的 `reference_filter` 标准逻辑 ， 严格修正了判定条件 ， 确保语义完全一致。
- 编译选项测试： 验证了去掉 `-march=native` 反而更慢 ， 在此场景下指令集优化是有正面收益的 ， 未产生指令膨胀的负面影响。

7. 实验收获（这个真是手打的了）

这一周是我经历最漫长的一周，但也是进步最快的一周（在这个方向），我不敢说我都学完了所有，这个关卡我也没有完全明白，但是如果没有这次的考核，我可能还在摆弄我的Python基础（现在才学到‘类’）,什么叫不能闭门造车，要多交流，现在我是意识到了。

我感觉我学到加速比，cpp中一些条件的判断也可以影响加速比，还有缓存命中越高越快啥的，还有一些正确性检验，其实还是感觉学得迷迷糊糊-.-’,至少知道一些以后可以学习的方向了，这倒是走出一些舒服圈了